TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Thanh Tuấn - 24122025

GIẢM CHIỀU DỮ LIỆU VỚI PCA

BÁO CÁO MÔN HỌC: PHƯƠNG PHÁP TOÁN CHO TRÍ TUỆ NHÂN TẠO
CHƯƠNG TRÌNH CHÍNH QUY

Tp. Hồ Chí Minh, tháng 04/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Thanh Tuấn - 24122025

GIẢM CHIỀU DỮ LIỆU VỚI PCA

BÁO CÁO MÔN HỌC: PHƯƠNG PHÁP TOÁN CHO TRÍ TUỆ NHÂN TẠO
CHƯƠNG TRÌNH CHÍNH QUY

GIÁO VIÊN HƯỚNG DẪN

TS. Cấn Trần Thành Trung
TS. Đỗ Đức Hào
Ths. Nguyễn Ngọc Toàn
CN. Trần Hoàng Quân

Tp. Hồ Chí Minh, tháng 04/2026

Mục lục

Mục lục i

1 Lý thuyết 1

1.1 Xây dựng bài toán 1

1.2 Các bước suy diễn toán học 1

1.3 Kết luận 3

1.4 Chứng minh 3

2 Lập trình cơ bản 4

3 Trực quan hóa 5

4 So sánh 6

4.1 So sánh 6

4.2 Đánh giá 7

5 Bài toán ứng dụng - nâng cao 8

5.1 Trực quan hóa 8

5.2 Phân tích 8

6 Mở rộng 10

Danh sách hình

1 Kết quả PCA 5
2 Trực quan hóa PCA trên bộ dữ liệu Iris 6
3 So sánh kết quả PCA (Tự cài đặt) và PCA (Sklearn) . . . 7
4 Đánh giá kết quả PCA (Tự cài đặt) và PCA (Sklearn) . . 7
5 Trực quan hóa ảnh gốc và ảnh sau PCA với các k khác nhau 9

1 Lý thuyết

1.1 Xây dựng bài toán

- **Bài toán giảm chiều dữ liệu - Principal Component Analysis (PCA)**: Giả sử ta có một tập dữ liệu X gồm n vector nằm trong không gian d chiều ($\vec{x}_i \in \mathbb{R}^d$). Mục tiêu của PCA là tìm cách chiếu tập dữ liệu này xuống một không gian mới có số chiều $k << d$ sao cho lượng thông tin mất đi trong quá trình giảm chiều là ít nhất.

- Trong PCA, thông tin được biểu diễn thông qua sự phân tán của dữ liệu. Do đó, mục tiêu cốt lõi là tìm ra một không gian chiếu mới sao cho **phương sai của dữ liệu trên không gian đó đạt giá trị tối đa**.

1.2 Các bước suy diễn toán học

- Trước khi thực hiện PCA, ta cần chuẩn hóa dữ liệu để đảm bảo rằng mỗi đặc trưng có cùng đơn vị đo lường và không bị chi phối bởi các đặc trưng có giá trị lớn hơn.

$$\vec{\mu} = \frac{1}{n}\sum_{i=1}^{n}\vec{x}_i$$
$$\hat{X} = X - \vec{\mu}$$

  Sau bước này, kỳ vọng của tập dữ liệu mới sẽ bằng 0.

- Ma trận hiệp phương sai (Covariance matrix) biểu diễn sự tương quan và độ phân tán của các chiều dữ liệu. Với tập dữ liệu đã được chuẩn hóa, ma trận hiệp phương sai Σ kích thước $d \times d$ được tính bằng công thức:

$$\Sigma = \frac{1}{n}\hat{X}\hat{X}^T$$

- Giả sử $\vec{\omega}$ là một vector đơn vị trong không gian d chiều ($||\vec{\omega}|| = 1$).

Phương sai của dữ liệu khi chiếu lên vector $\vec{\omega}$ được tính bằng:

$$
\begin{aligned}
Var(\hat{X}) &= \frac{1}{n}\sum_{i=1}^{n} ||proj_{\vec{\omega}}(\vec{x}_i) - \mu||^2 = \frac{1}{n}\sum_{i=1}^{n} ||proj_{\vec{\omega}}(\vec{x}_i)||^2 \\
&= \frac{1}{n}\sum_{i=1}^{n} proj_{\vec{\omega}}(\vec{x}_i) \cdot proj_{\vec{\omega}}(\vec{x}_i) \\
&= \frac{1}{n}\sum_{i=1}^{n} (\vec{x}_i \cdot \vec{\omega})\vec{\omega} \cdot (\vec{x}_i \cdot \vec{\omega})\vec{\omega} \\
&= \frac{1}{n}\sum_{i=1}^{n} (\vec{x}_i \cdot \vec{\omega})^2 (\vec{\omega} \cdot \vec{\omega}) \\
&= \frac{1}{n}\sum_{i=1}^{n} (\vec{x}_i \cdot \vec{\omega})^2 \\
&= \frac{1}{n}\vec{\omega}^T \hat{X}\hat{X}^T \vec{\omega} \\
&= \vec{\omega}^T \Sigma \vec{\omega}
\end{aligned}
$$

Ta muốn tìm vector $\vec{\omega}$ sao cho $Var(\hat{X})$ đạt giá trị tối đa với điều kiện $||\vec{\omega}|| = 1$. Đây là một bài toán tối ưu hóa có ràng buộc, và ta có thể sử dụng phương pháp Lagrange để giải quyết:

$$L(\vec{\omega}, \lambda) = \vec{\omega}^T \Sigma \vec{\omega} - \lambda(\vec{\omega}^T \vec{\omega} - 1)$$

Để tìm cực trị của hàm Lagrange, ta tính đạo hàm riêng theo $\vec{\omega}$ và λ và đặt chúng bằng 0:

$$\frac{\partial L}{\partial \vec{\omega}} = 2\Sigma\vec{\omega} - 2\lambda\vec{\omega} = 0 \tag{1}$$

$$\frac{\partial L}{\partial \lambda} = -(\vec{\omega}^T \vec{\omega} - 1) = 0 \tag{2}$$

Từ phương trình (1.1), ta có:

$$\Sigma\vec{\omega} = \lambda\vec{\omega}$$

1.3 Kết luận

Qua phép biến đổi toán học bằng hàm Lagrange, ta thấy rằng hướng chiếu $\vec{\omega}$ tối ưu hóa phương sai thỏa mãn phương trình $\Sigma\vec{\omega} = \lambda\vec{\omega}$. Điều này giải thích vì sao việc giải bài toán PCA hoàn toàn tương đương với việc tìm các vector riêng (eigenvectors) của ma trận hiệp phương sai Σ.

Eigenvalues có ý nghĩa rằng, nếu ta nhân hai vế của phương trình $\Sigma\vec{\omega} = \lambda\vec{\omega}$ với $\vec{\omega}^T\vec{\omega} = 1$, ta có $\vec{\omega}^T\Sigma\vec{\omega} = \vec{\omega}^T\lambda\vec{\omega} = \lambda(\vec{\omega}^T\vec{\omega}) = \lambda$. Vì $\vec{\omega}^T\Sigma\vec{\omega}$ chính là phương sai của dữ liệu trên hướng $\vec{\omega}$, ta kết luận rằng trị riêng (eigenvalue) λ chính là lượng phương sai (thông tin) được giữ lại trên thành phần chính tương ứng.

Để tối ưu hóa việc giảm chiều, ta sẽ tính toán tất cả các vector riêng và trị riêng, sau đó sắp xếp chúng theo thứ tự trị riêng giảm dần ($\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_k$). Các vector riêng ứng với các trị riêng lớn nhất sẽ là các thành phần chính quan trọng nhất. Ta sẽ chọn k vector riêng đầu tiên để tạo thành ma trận chiếu.

1.4 Chứng minh

Như đã phân tích ở phần kết luận, phương sai của dữ liệu khi chiếu lên hướng của vector riêng $\vec{\omega}$ có giá trị bằng đúng trị riêng λ tương ứng ($Var = \lambda$). Do mục tiêu của PCA là tối đa hóa phương sai ($\max \vec{\omega}^T\Sigma\vec{\omega}$), ta cần chọn giá trị λ lớn nhất có thể. Do đó, **hướng $\vec{\omega}$ làm phương sai đạt cực đại bắt buộc phải là vector riêng ứng với trị riêng lớn nhất**.

Chứng minh tính trực giao của các principal components: Giả sử $\vec{\omega}_1$ và $\vec{\omega}_2$ là hai vector riêng của ma trận hiệp phương sai Σ với các trị riêng khác nhau λ_1 và λ_2. Ta có:

$$\Sigma\vec{\omega}_1 = \lambda_1\vec{\omega}_1$$
$$\Sigma\vec{\omega}_2 = \lambda_2\vec{\omega}_2$$

Ta có:

$$\Sigma\vec{\omega}_1 \cdot \vec{\omega}_2 = \lambda_1\vec{\omega}_1 \cdot \vec{\omega}_2 \tag{3}$$

Mặt khác,

$$\Sigma\vec{\omega}_1 \cdot \vec{\omega}_2 = \vec{\omega}_1 \cdot (\Sigma^T\vec{\omega}_2) = \vec{\omega}_1 \cdot (\Sigma\vec{\omega}_2) = \lambda_2\vec{\omega}_1 \cdot \vec{\omega}_2 \tag{4}$$

Từ (3), (4) và $\lambda_1 \neq \lambda_2$, ta suy ra $\vec{\omega}_1 \cdot \vec{\omega}_2 = 0$, tức là $\vec{\omega}_1$ và $\vec{\omega}_2$ là trực giao với nhau.

2 Lập trình cơ bản

Từ section này trở đi, để thuận tiện cho việc code, ta sẽ định nghĩa các vector là vector hàng thay vì vector cột như trong phần lý thuyết. Khi đó, ma trận dữ liệu X sẽ có kích thước $n \times d$, và ma trận hiệp phương sai Σ sẽ được tính bằng công thức $\Sigma = \frac{1}{n}X^TX$ thay vì $\Sigma = \frac{1}{n}XX^T$ như trong phần lý thuyết.

Ở câu này ta sẽ viết chương trình thực hiện PCA từ đầu (không dùng thư viện PCA có sẵn). Để thực hiện PCA, ta sẽ thực hiện các bước sau:

- Chuẩn hóa dữ liệu.
- Tính ma trận hiệp phương sai.
- Tính các vector riêng và trị riêng của ma trận hiệp phương sai.
- Sắp xếp các vector riêng theo thứ tự trị riêng giảm dần và chọn k vector riêng đầu tiên để tạo thành ma trận chiếu.
- Chiếu dữ liệu gốc lên không gian con được tạo bởi các vector riêng đã chọn để thu được dữ liệu đã giảm chiều.

Đối với thư viện PCA có sẵn, các vector thường là các vector hàng, khác với các định nghĩa toán học trong đó các điểm dữ liệu là các vector cột. Do đó đầu vào của thuật toán được lập trình sẽ là một ma trận X có kích thước $n \times d$, trong đó mỗi hàng đại diện cho một điểm dữ liệu và mỗi cột đại diện cho một đặc trưng. Kết quả đầu ra sẽ là một ma trận Y có kích thước $n \times k$, trong đó mỗi hàng đại diện cho một điểm dữ liệu đã được chiếu lên không gian con k chiều.

Các giá trị của eigenvalues, eigenvectors và ma trận chứa dữ liệu sau khi chiếu sang không gian mới được thể hiện trong Hình 1.

```
(.venv) C:\University\M4AI\Lab2>c:/University/M4AI/Lab2/.venv/Scripts/python.exe c:/University/M4AI/Lab2/cau2.py
Original Data:
 [[ 0.49671415 -0.1382643   0.64768854  1.52302986 -0.23415337]
 [-0.23413696  1.57921282  0.76743473 -0.46947439  0.54256004]
 [-0.46341769 -0.46572975  0.24196227 -1.91328024 -1.72491783]
 [-0.56228753 -1.01283112  0.31424733 -0.90802408 -1.4123037 ]
 [ 1.46564877 -0.2257763   0.0675282  -1.42474819 -0.54438272]
 [ 0.11092259 -1.15099358  0.37569802 -0.60063869 -0.29169375]
 [-0.60170661  1.85227818 -0.01349722 -1.05771093  0.82254491]
 [-1.22084365  0.2088636  -1.95967012 -1.32818605  0.19686124]
 [ 0.73846658  0.17136828 -0.11564828 -0.3011037  -1.47852199]
 [-0.71984421 -0.46063877  1.05712223  0.34361829 -1.76304016]]
Eigenvalues: [1.71341075 1.22995593 0.21660019 0.58181577 0.44971967]
Eigenvectors:
 [[ 0.19140105 -0.24462942 -0.16210015 -0.908652    0.2271501 ]
 [-0.64514401 -0.30162525 -0.53097574 -0.07329169 -0.45332605]
 [ 0.30640856 -0.45450065  0.47826195 -0.06907329 -0.68266928]
 [ 0.22650162 -0.75905736 -0.23943367  0.3946124   0.39935136]
 [-0.63400762 -0.25775906  0.63696572 -0.09214015  0.34260688]]
Projected Data:
 [[ 0.64155188 -2.03803058]
 [-1.51341026 -1.11948201]
 [ 0.71155224  1.47263514]
 [ 1.09722879  0.78536137]
 [ 0.23471001  0.33251564]
 [ 0.69319364  0.11224777]
 [-2.30996327 -0.38265634]
 [-1.62912653  1.51561675]
 [ 0.62994299 -0.13825698]
 [ 1.44432052 -0.53995076]]

(.venv) C:\University\M4AI\Lab2>
```

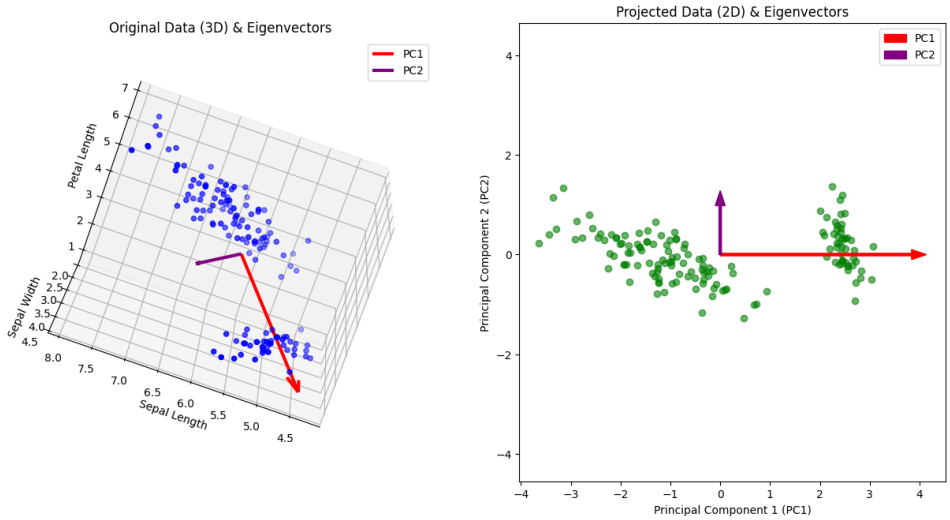
Hình 1: Kết quả PCA

3 Trực quan hóa

Dataset lựa chọn là bộ dữ liệu Iris, sử dụng các đặc trưng gồm: Sepal Length, Sepal Width, Petal Length. Kết quả sau khi thực hiện PCA trên bộ dữ liệu và trực quan hóa tại Hình 2.

Phân tích:

- So sánh dữ liệu trước và sau PCA:
  - Trước PCA, dữ liệu nằm trong không gian 3 chiều, khó trực quan hóa hoặc không thể trực quan hóa (đối với dữ liệu nằm trong không gian nhiều hơn 3 chiều) cũng như khó phân biệt các lớp.
  - Sau PCA, dữ liệu được chiếu xuống không gian 2 chiều, giúp dễ dàng trực quan hóa và phân biệt các lớp hơn.
- Nhận xét về phương sai:



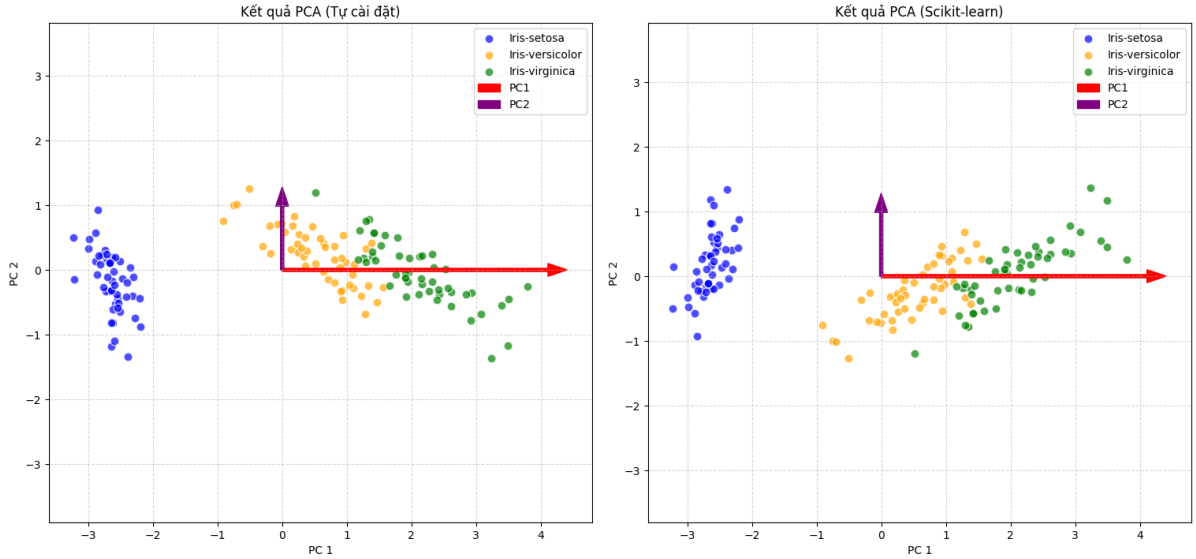
Hình 2: Trực quan hóa PCA trên bộ dữ liệu Iris

- Trên các đồ thị trực quan hóa, PC1 luôn có phương sai lớn nhất, đại diện cho hướng chiếu giữ lại nhiều thông tin nhất về sự phân tán của dữ liệu. PC2 có phương sai nhỏ hơn, nhưng vẫn giữ lại một phần thông tin quan trọng.
- Việc giảm chiều dữ liệu từ 3 chiều xuống 2 chiều đã làm mất đi một phần thông tin, tuy nhiên tổng phương sai của PC1 và PC2 vẫn giữ lại phần lớn thông tin quan trọng (đối với bộ dữ liệu này là 98.51% - xem trong kết quả của code câu 3).

4 So sánh

4.1 So sánh

Từ kết quả của Hình 3 và 4 ta thấy rằng, giá trị của eigenvalues và phương sai giữ lại của PCA tự cài đặt và PCA sử dụng thư viện Sklearn là hoàn toàn giống nhau. Điều này chứng tỏ rằng thuật toán PCA được cài đặt từ đầu đã hoạt động chính xác và cho kết quả tương đương với phiên bản đã được tối ưu hóa trong thư viện Sklearn.



Hình 3: So sánh kết quả PCA (Tự cài đặt) và PCA (Sklearn)

```
(.venv) C:\University\M4AI\Lab2>c:/University/M4AI/Lab2/.venv/Scripts/python.exe c:/University/M4AI/Lab2/cau4.py
--- So sánh Eigenvalues---
Tự cài đặt: [4.22484077 0.24224357]
Scikit-learn: [4.22484077 0.24224357]

--- Đánh giá tốc độ chạy ---
Tự cài đặt: 0.000333 giây
Scikit-learn: 0.000612 giây

--- Đánh giá sai số giữa 2 cách ---
Sai số trung bình tuyệt đối: 0.0000000000

(.venv) C:\University\M4AI\Lab2>
```

Hình 4: Đánh giá kết quả PCA (Tự cài đặt) và PCA (Sklearn)

4.2 Đánh giá

- Các trị riêng có thể bị lật dấu tùy vào thuật toán giải eigenvalue được sử dụng, do đó để đánh giá sai số giữa hai cách chạy, ta lấy trị tuyệt đối của các trị riêng, sau đó tính sai số tuyệt đối trung bình giữa hai cách. Kết quả cho thấy rằng sai số xấp xỉ bằng 0.

- Về thời gian chạy, thuật toán PCA tự cài đặt có thời gian chạy ngắn hơn so với PCA sử dụng thư viện Sklearn. Điều này có thể đến từ các lý do sau:

  – Bộ dữ liệu nhỏ, chỉ gồm 3 đặc trưng.

- PCA tự cài đặt có thể sử dụng một thuật toán giải eigenvalue đơn giản hơn, trong khi Sklearn có thể sử dụng một thuật toán phức tạp hơn (như SVD) để đảm bảo độ chính xác cao hơn.
- PCA của thư viện sklearn có thể có một vài thao tác ngầm như kiểm tra tính hợp lệ của dữ liệu, khởi tạo đối tượng, lựa chọn thuật toán,... Trong khi PCA tự cài đặt có thể bỏ qua một số bước này.

5 Bài toán ứng dụng - nâng cao

5.1 Trực quan hóa

Ảnh được chọn là "image_0032.jpg" trong thư mục "panda" từ bộ dữ liệu caltech-101 dataset. Lỗi tái tạo (reconstruction error) có nhiều cách đo lường, trong đó Mean Squared Error (MSE) là một trong những phương pháp phổ biến nhất. MSE được tính bằng công thức:

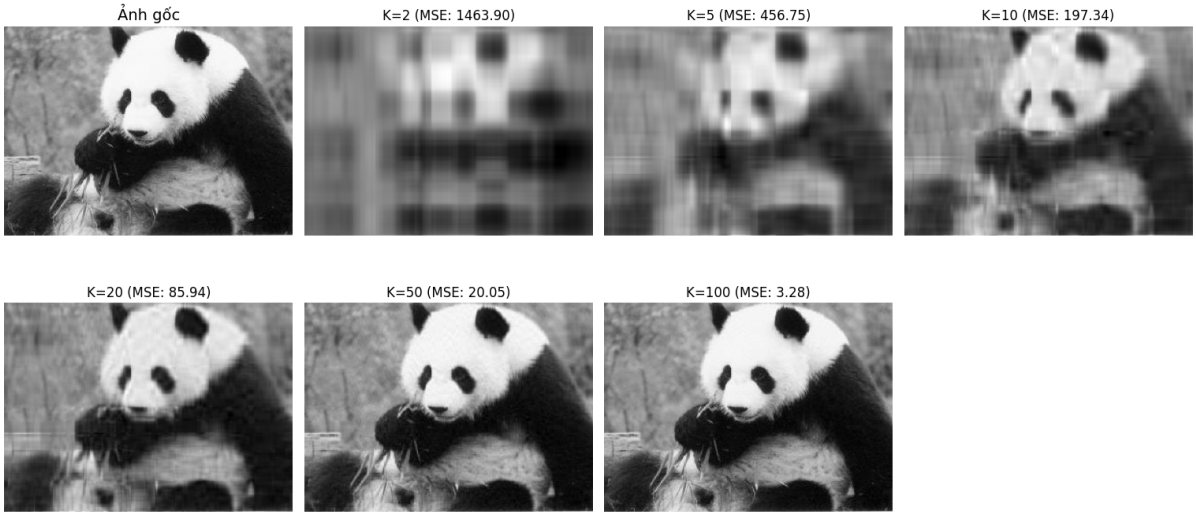
$$MSE = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$$

Trong đó x_i là giá trị pixel gốc của ảnh, $\hat{x}_i$ là giá trị pixel sau khi tái tạo từ PCA, và n là tổng số pixel trong ảnh.

Ảnh gốc và ảnh sau PCA với các k khác nhau được thể hiện ở Hình 5.

5.2 Phân tích

- Ảnh hưởng của số lượng thành phần chính đối với chất lượng ảnh:
  - Với k nhỏ (ví dụ k = 2, 5, 10, 20), ảnh sau PCA có chất lượng kém, mất nhiều chi tiết và trông mờ hơn so với ảnh gốc. Điều này là do khi giảm chiều dữ liệu quá nhiều, ta đã loại bỏ một phần lớn thông tin quan trọng về ảnh (như vân bề mặt, nếp gấp, hay viền cạnh), dẫn đến việc tái tạo ảnh không chính xác. Đồng thời hình ảnh xuất hiện nhiều nhiễu dạng sọc.



Hình 5: Trực quan hóa ảnh gốc và ảnh sau PCA với các k khác nhau

  - Với k lớn hơn (ví dụ k = 50, 100), ảnh sau PCA có chất lượng tốt hơn, gần giống với ảnh gốc hơn. Lý do là khi giữ lại nhiều thành phần chính hơn, ta đã giữ lại nhiều thông tin quan trọng về ảnh, giúp tái tạo ảnh chính xác hơn.

- Sự đánh đổi giữa độ nén và chất lượng ảnh:
  - Việc đánh đổi là một điều tất yếu. Trong quá trình nén ảnh, một phần thông tin sẽ bị mất đi, và điều này ảnh hưởng trực tiếp đến chất lượng ảnh sau khi giải nén. Dựa theo kết quả của k và MSE, ta có thể thấy rằng khi k tăng lên, MSE giảm xuống, cho thấy chất lượng ảnh được cải thiện. Tuy nhiên, việc tăng k cũng đồng nghĩa với việc giữ lại nhiều thành phần chính hơn, dẫn đến kích thước dữ liệu lớn hơn và giảm khả năng nén.
  - Việc lựa chọn k phù hợp phụ thuộc vào mục tiêu sử dụng ảnh sau PCA. Nếu mục tiêu là nén ảnh để tiết kiệm không gian lưu trữ hoặc băng thông, ta có thể chấp nhận một mức độ mất mát chất lượng nhất định và chọn k nhỏ hơn. Ngược lại, nếu mục tiêu là tái tạo ảnh với chất lượng cao, ta nên chọn k lớn hơn để giữ lại nhiều thông tin hơn.

6 Mở rộng

Các nội dung ở mục mở rộng gồm:

- So sánh PCA với t-SNE, LDA.
- Áp dụng PCA cho giảm nhiễu tín hiệu.

Có thể tóm tắt ý tưởng của các thuật toán t-SNE, LDA như sau:

- t-SNE: là một thuật toán giảm chiều phi tuyến tính hoạt động dựa trên nguyên lý bảo toàn cấu trúc lân cận. Quá trình bắt đầu bằng việc đo lường mức độ tương đồng giữa các điểm trong không gian đa chiều gốc để lập nên một "bản đồ" xác suất. Sau đó, các điểm dữ liệu được khởi tạo vị trí ngẫu nhiên xuống một không gian ít chiều hơn (thường là 2D). Thông qua việc tối ưu hóa một hàm mất mát, thuật toán liên tục điều chỉnh vị trí của các điểm ở không gian mới cho đến khi cấu trúc phân cụm khớp hoàn hảo với bản đồ gốc.
- LDA: là một thuật toán giảm chiều dữ liệu có giám sát. Ý tưởng cốt lõi của thuật toán là tìm kiếm một không gian chiếu tuyến tính sao cho khoảng cách giữa tâm của các cụm dữ liệu (phân lớp) được kéo ra xa nhất có thể, đồng thời tối thiểu hóa phương sai nội bộ bên trong từng cụm.

Chi tiết code và kết quả sẽ được trình bày tại file "bonus.ipynb" trong GitHub repo này.